

APPENDICES

Quick-reference materials for daily use

Appendix A: Quick-Reference Prompt Library

The prompts below are the highest-utility templates from this book. Replace every [bracketed item] with your values. Full library also at the downloadable resources repository.

A.1 Five-Component Prompt Structure

- 1. **ROLE:** Act as a senior [language/framework] developer.
- 2. **CONTEXT:** This is a [framework] project using [stack].
The relevant file is [filename].
- 3. **TASK:** [Specific imperative task description]
- 4. **CONSTRAINTS:** Follow [conventions]. Do not [anti-patterns].
- 5. **SUCCESS:** [Expected output format / acceptance criteria]

A.2 Feature Scaffold

- > Scaffold a new feature: [feature name and description].
Create files in order: schema → queries → server actions
→ UI components → tests. Follow all CLAUDE.md conventions.
Show me a file list before writing any code.

A.3 Test Generation

- > Write a complete test suite for [filename].
Framework: [Jest/Vitest/pytest]. Mock all external deps.
Cover: happy path, all error cases, null/empty edge cases.
Test behavior, not implementation. Target 90%+ coverage.

A.4 Debugging Protocol

- > Debug this error: [paste full error + stack trace].
Step 1: Root cause – not the symptom.
Step 2: Minimal reproduction.
Step 3: The fix.
Step 4: A test that would have caught this bug.

A.5 Audit → Protect → Transform (Refactoring)

- Phase 1 – Audit: Analyze [function]. List responsibilities, dependencies, assumptions, danger zones. No code changes yet.
- Phase 2 – Protect: Write characterization tests. Mark known bugs: // CHARACTERIZATION: known quirk
- Phase 3 – Transform: Refactor step by step. Tests green each step.

A.6 PR Review (/review-pr)

- > Review the git diff of this branch against main.
Check: (1) correctness/edge cases, (2) security – inputs, auth, error messages, (3) CLAUDE.md conventions, (4) test coverage, (5) performance (N+1, indexes).
Format: Critical → Major → Minor → Suggestions → Verdict.

A.7 Safe Deploy (use before any infra prompt)

- > Generate [Dockerfile / Actions / Terraform] FOR REVIEW ONLY.
I will apply to real infrastructure myself after reviewing.
Do not run apply, push to main, or modify production resources.

A.8 MCP Multi-Tool Workflow

- > Implement the feature in GitHub issue #[N].
 - 1. Read issue + comments (GitHub MCP).
 - 2. Read relevant database schema (Postgres MCP).

3. Verify current best practices (web search MCP).
4. Present implementation plan. Wait for approval.
5. Implement after approval.